JCEC
THE JOURNAL OF ENGINEERING AND EXACT SCIENCES

OPEN ACCESS

The Journal of Engineering and Exact Sciences – jCEC, Vol. 11 N. 01 (2025)
journal homepage: https://periodicos.ufv.br/jcec
eISSN: 2527-1075
ISSN: 2446-9416

Post-Quantum Cryptography for the Internet of Things: new approach

Criptografia Pós-Quântica para a Internet das Coisas: nova abordagem

Article Info:
Article history: Received 2025-02-16 / Accepted 2025-06-01 / Available online 2025-06-01
doi: 10.18540/jcecvl11iss1pp21741

Pablo Rubens de Moura
ORCID: https://orcid.org/0009-0002-9637-8421
Course: Interdisciplinar em Ciencia e Tecnologia (C&T)
Departamento de Ciências Naturais, Matemática e Estatística DCME/CCEN
Universidade Federal Rural do Semiárido UFERSA, Mossoró/RN, Brazil
E-mail: pablomoura164@gmail.com
Elmer Rolando Llanos Villarreal
ORCID: https://orcid.org/0000-0002-3059-3340
Departamento de Ciências Naturais, Matemática e Estatística DCME/CCEN
Universidade Federal Rural do Semiárido UFERSA, Mossoró/RN, Brazil
E-mail: elmerllanos@ufersa.edu.br
Diego Antonio de Moura Fonsêca
ORCID: https://orcid.org/0000-0003-0895-3300
Departamento de Engenharia de Computação e Automação (DCA)
Universidade Federal do Rio Grande do Norte UFRN, Brazil
Email: diegomoura@dca.ufn.br
Andrés Ortiz Salazar
ORCID: https://orcid.org/0000-0001-5650-3668
Departamento de Engenharia de Computação e Automação (DCA)
Universidade Federal do Rio Grande do Norte UFRN, Brazil
Email: andres@dca.ufn.br

Abstract
The growth of quantum computing reveals a series of significant threats to cryptographic systems known today, such as RSA, ECC, and Diffie-Hellman, which are extremely important for Internet security, but are threatened by the increase in possible quantum attacks. The area of public and private key cryptography is vast. Therefore, this article addresses post-quantum cryptography (PQC) algorithms and the challenges of implementing them in Internet of Things (IoT) devices, given the difficulty of application due to restricted resources, such as RAM and clock speed. To this end, we will discuss the need for standardization of algorithms considering low-resource IoT devices, prioritizing RAM over ROM, meeting the needs of low-memory devices, which are lighter. The National Institute of Standards and Technology (NIST) has validated four cryptographic systems that are secure against quantum attacks. We analyze the implications of these systems on the infrastructure of current networks and will compare in detail these algorithms and their respective strengths, weaknesses, and trade-offs. We evaluate the feasibility of post-quantum encryption (PQC) algorithms in resource-constrained IoT devices using the ESP-WROOM-32 microcontroller. Three cryptographic schemes were implemented and compared.
Keywords: Efficiency. Post-Quantum. IoT. Microcontrollers. Protocols.

Resumo
O crescimento da computação quântica revela uma série de ameaças significativas aos sistemas criptográficos conhecidos hoje, como RSA, ECC e Diffie-Hellman, que são extremamente importantes para a segurança da internet, mas são ameaçados pelo aumento em possíveis ataques quânticos. A área da criptografia de chaves públicas e privadas é vasta. Portanto, este artigo aborda algoritmos de criptografia pós-quântica (PQC) e os desafios de implementá-los em dispositivos de Internet das Coisas (IoT), dada a dificuldade de aplicação devido a recursos restritos, como RAM e velocidade de clock. Para este fim, discutiremos a necessidade de padronização de algoritmos considerando dispositivos IoT de baixo recurso, priorizando RAM sobre ROM, atendendo às necessidades de dispositivos de baixa memória, que são mais leves. O Instituto Nacional de Padrões e Tecnologia (NIST) validou quatro sistemas criptográficos que são seguros contra ataques quânticos. Analisamos as implicações desses sistemas na infraestrutura das redes atuais e compararemos em detalhes esses algoritmos e suas respectivas forças, fraquezas e trade-offs. Avaliamos a viabilidade de algoritmos de criptografia pós-quântica (PQC) em dispositivos IoT com recursos limitados usando o microcontrolador ESP-WROOM-32. Três esquemas criptográficos foram implementados e comparados.
Keywords: Eficiência. Pós-Quântico. IoT. Microcontroladores. Protocolos

Nomenclature

PQC - Post-Quantum Cryptography (Criptografia Pós-Quântica)

IoT - Internet of Things (Internet das Coisas)

NIST - National Institute of Standards and Technology (Instituto Nacional de Padrões e Tecnologia)

RSA - Rivest-Shamir-Adleman (Sistema de criptografia de chave pública)

ECC - Elliptic Curve Cryptography (Criptografia de Curvas Elípticas)

DoS - Denial of Service (Negação de Serviço)

MQTT - Message Queuing Telemetry Transport (Protocolo de mensagens para IoT)

TLS - Transport Layer Security (Segurança da Camada de Transporte)

RAM - Random Access Memory (Memória de Acesso Aleatório)

ROM - Read-Only Memory (Memória Somente de Leitura)

1. INTRODUCTION

The Internet of Things (IoT) is growing exponentially, with billions of interconnected devices exchanging information in real-time (Selvaraj and Uddin, 2023). This expansion drives applications in various fields, from smart manufacturing to remote healthcare (Zhang *et al.*, 2023). However, the rapid growth of IoT has not been accompanied by an equivalent improvement in its security mechanisms, given the increasing threat posed by quantum computing algorithms to traditional cryptographic systems. As a result, contemporary IoT systems are exposed to the risk of sensitive information leaks, potentially leading to a loss of confidentiality and integrity (Pursche *et al.*, 2024).

These devices encompass a vast network of interconnected and communicative equipment, ranging from simple sensors — such as those used in smart homes to monitor temperature and lighting — to complex machinery in industrial environments (Liu *et al.*, 2024). The defining characteristic of an IoT device is its ability to collect and exchange data, often in real time, over the internet (Madhu and Gias, 2023; Tanweer *et al*., 2018). According to data from Statista (Tanweer, 2018), the number of devices connected to the Internet is constantly increasing, reaching 25 billion

in 2022 and projected to reach 75 billion by 2025, as shown in Figure 1. While this rapid growth is promising, it raises serious security concerns. The growing number of devices aligns with the increasing availability of quantum computing hardware (Anjan *et al.*, 2024). Projections indicate that the processing power of these systems will reach a level where current encryption schemes will become vulnerable within a reasonable time frame (Policarpo, 2023).

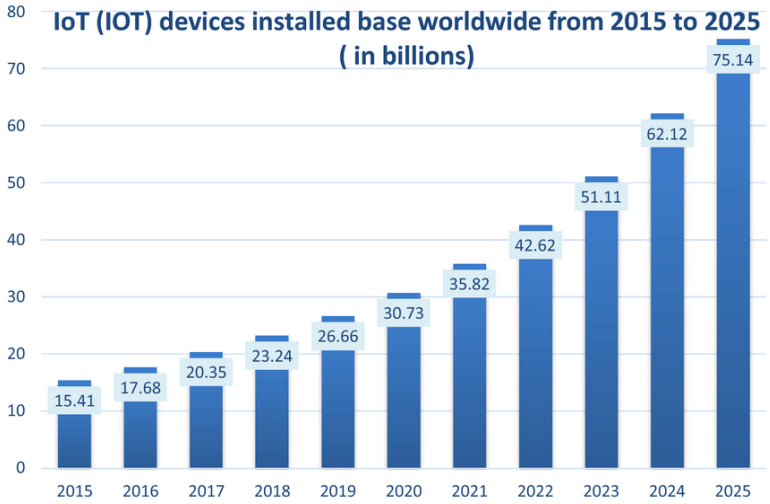

Figure 1 - Number of connected IoT devices worldwide from 2015 to 2025, in billions (Tanweer, 2018).

Quantum computing threatens the most common public-key cryptographic systems. Quantum attacks can be applied to other widely used public-key algorithms based on integer factorization and discrete logarithm problems, such as RSA, ECDSA, ECDH, DSA, and others relying on these same mathematical foundations (Bharat *et al.*, 2024). All of these can be efficiently solved using Shor's algorithm on a sufficiently powerful quantum computer (Kwala, Kant, & Mishra, 2024).

Quantum computers can also accelerate brute-force attacks against symmetric ciphers by approximately a quadratic factor using known algorithms such as Grover's algorithm. Although post-quantum cryptographic (PQC) algorithms are theoretically secure against quantum attacks, they are computationally more complex than their classical counterparts. Additionally, they face resource constraints such as memory, bandwidth, and processing power in IoT devices, increasing the complexity of their implementation. The increased complexity demands software and hardware optimization strategies to enhance the performance and energy efficiency of PQC algorithms on embedded hardware platforms. This can be achieved through techniques involving the application of mathematical concepts in software and hardware optimizations (Imine & Ali-Pacha, 2022).

Given this scenario, the efficient implementation of PQC algorithms in IoT devices becomes crucial. This study aims to assess the hardware limitations of current IoT devices, consider the complexity of data usage, and improve the runtime performance of PQC algorithms while minimizing resource dependency. The objective is to propose both practical and theoretical strategies for software and hardware optimization. We plan to analyze trade-offs between security, performance, and energy consumption, compare optimized approaches with existing algorithms, and explore opportunities in the architecture and logical implementation of cryptographic algorithms. Through a detailed literature review and empirical testing, we aim to provide guidelines and best practices, contributing to a secure transition to PQC for IoT devices. This paper analyzes post-quantum cryptographic (PQC) algorithms and the challenges of implementing them in Internet of Things (IoT) devices, given the difficulty of application due to constrained resources such as RAM and clock speed.

The paper is structured as follows: Section 2 presents fundamental cryptographic principles. Section 3 discusses IoT architecture. Section 4 introduces post-quantum cryptographic algorithms. Section 5 evaluates their performance and efficiency. Section 6 examines post-quantum IoT security protocols. Section 7 presents a case study on PQC implementation in a microcontroller. Section 8 provides performance assessments. Section 9 presents results and discussions. Section 10 concludes the paper.

2. PRINCIPLES OF CRYPTOGRAPHY

According to Liu *et al.* (2024), cryptography is the science that uses mathematics to encrypt and decrypt data. It enables confidential information to be stored or transmitted over insecure networks, rendering it unreadable to anyone other than the intended recipient. A cryptographic algorithm or cipher is a mathematical function used in the encryption and decryption process. Cryptographic algorithms work with keys—such as words, numbers, or phrases—to encrypt plaintext, ensuring that the same plaintext is encrypted into different ciphertexts when different keys are used. The security of encrypted data depends entirely on two factors: the strength of the cryptographic algorithm and the confidentiality of the key.

Aryan *et al.*, (2023) says that modern cryptography is divided into two categories based on key usage: symmetric and asymmetric (also known as public-key cryptography). In symmetric cryptography, the same key is used for both encryption and decryption. In asymmetric cryptography, a "private key" is kept secret, while a "public key" is made available to others. This arrangement enables unique security features: when the public key is used for encryption, it ensures that only the intended recipient can read a secret message when the private key is used for encryption, any entity with the corresponding public key can verify that a non-repudiable message was created by the private key owner.

In symmetric cryptography, the same key is used to both encrypt and decrypt data. This approach is efficient for large amounts of data but requires secure key sharing between communicating parties. Symmetric ciphers operate through permutation, substitution, and other transformations designed to "scramble" plaintext into ciphertext that can only be recovered with the correct key. One such example is AES, which can be used in operational modes such as GCM (Aryan *et al.*, 2023).

Asymmetric cryptography, on the other hand, uses a pair of keys: a private key and a public key, which can be freely distributed. The public key is used to encrypt messages that can only be decrypted with the corresponding private key. The private key can also be used to sign messages, allowing any entity with the corresponding public key to verify the signature, ensuring confidentiality and authenticity. This mechanism is essential for authentication and secure key exchange. The mathematical functions underlying asymmetric cryptography are more complex than those used in symmetric cryptography, making their computations more intensive. A prime example is RSA, which is based on the factorization of prime numbers (Aryan *et al.*, 2023).

In addition to these cryptographic methods, modern cryptography also employs hash functions—algorithms that transform data of any size into a fixed-size value, known as a hash or message digest. Cryptographic hash functions, such as SHA-1, SHA-224, SHA-256, and SHA-384, are designed to be one-way functions, making it computationally infeasible to reconstruct the original data from the hash. They are also resistant to collisions, meaning it is extremely difficult to find two different sets of data that produce the same hash (Bharat *et al.*, 2024).

3. IOT ARCHITECTURE

IoT architecture is characterized by the interconnection of its components. Unlike traditional systems, IoT involves a myriad of interconnected devices that act as both sources and destinations of data (Tanweer *et al*, 2018). This architecture generally comprises three main layers, exemplified in Figure 2.

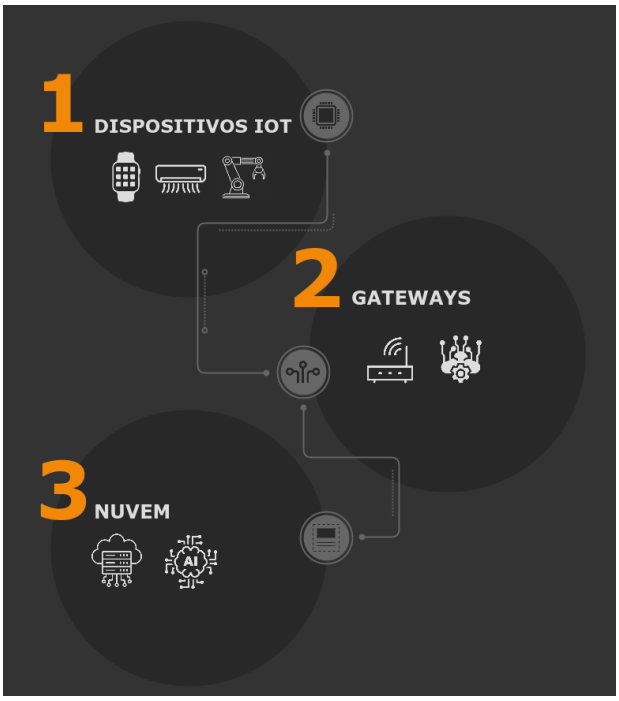

Figure 2 – Simplified and generalized IoT architecture.

- IoT Devices: These range from wearable actuators and simple sensors to complex actuators. They are responsible for collecting data and have internet connectivity, but they also operate with limited resources (Pursche *et al.*, 2024).
- Gateways: These function as intermediaries between IoT devices and the internet or cloud. Gateways connect low-power IoT nodes (Zigbee, LoRaWAN) to wide-area networks (TCP/IP) and may include devices such as routers or edge computing devices (Liu *et al.*, 2024).
- Cloud: This is where the collected data is stored, processed, and analyzed (Fernández-Caramés, 2019). The cloud provides scalable computing, AI, and ML for IoT analytics, utilizing remote servers, platforms, and data centers (Choi & Lee, 2024).

The physical layer, defined by Liu *et al.* (2024), consists of the first layer of these devices, often operates with communication protocols that lack robust cryptographic security. An example is MQTT-SN, which does not define encryption at the application layer by default, making it vulnerable to security threats in WiFi/802.11 or other protocols. Similarly, CoAP allows operation in NoSec mode, meaning it does not enforce encryption by default, exposing sensitive APIs (Roldán-Gómez *et al.*, 2022); (Silva, Cunha, Barraca, & Aguiar, 2023).

Atkins (2021) and Ilgın *et al.* (2024) say that implementing post-quantum cryptography (PQC) at this layer requires lattice-based algorithms that perform polynomial arithmetic on matrices, demanding up to 20kB of RAM—exceeding the capacity of some 8-bit microcontrollers with clock speeds measured in MHz rather than GHz (e.g., Arduino Uno has only 2kB of RAM). Alternatives such as hash-based cryptographic algorithms ensure secure information exchange against quantum computing threats, but they generate signatures of approximately 41kB, making them impractical for transmission over networks like LoRaWAN and Sigfox. These LPWAN technologies are designed for microtransactions of data (in bytes, not kilobytes).

Gateways, in turn, represent a Single Point of Failure (SPOF) due to their dual operational role: ensuring interoperability between different communication standards. When translating messages from LPWAN networks to HTTP/HTTPS, some gateways rely on TLS 1.2 or lower implementations, such as the AVNET-TO136 module, which depends on ciphers like RSA-2048 or ECDSA for authentication—both of which are vulnerable to quantum algorithms (Liu *et al.*, 2024). A cryptographic failure at the gateway level could result in catastrophic consequences for the entire infrastructure, as a compromised gateway exposes the entire network, from the gateway itself to edge computing applications.

Edge computing, on the other hand, brings data processing closer to the source (Ilgın *et al.*, 2024). This means that part of the processing typically performed in the cloud is executed closer to IoT devices, either on gateways or other edge devices. By reducing the need to send data to the cloud, edge computing minimizes latency, improves response time, and saves bandwidth (Flavio *et al.*, 2024). Gateways function as fog computing nodes, aggregating, filtering, and pre-processing data before sending it to the cloud. For example, an ARM Cortex-A53 gateway can run anomaly detection algorithms on telemetry streams, reducing the volume of data transmitted to the cloud by up to 70% (Kim *et al.*, 2022). The workload on a gateway can be modeled as follows:

$$W = \sum_{i=1}^{N}(D_i \times C_p) + \sum_{j=1}^{M}(P_j \times C_t) \quad (1)$$

where D_i is the device's data volume i, C_p is the computational cost of pre-processing, P_j is the number of protocol packets j, and C_t is the conversion cost. For N and M at high values, a gateway saturation becomes inevitable, creating bottlenecks. The cloud offers near-unlimited resources, while the edge operates under hardware constraints. The gateway, however, occupies an intermediate position. Modules like the NXP i.MX 8 combine ARM Cortex-A53 cores for general processing with dedicated accelerators, enabling tasks such as data compression using LZ77 algorithms or AES-GCM encryption without overloading the main CPU. ARM Cortex-M0/M3 typically have 8–64 KB, while more robust models like Cortex-M4 or M7 reach 128–512 KB.Unlike the cloud, which scales horizontally through clusters, gateways have fixed capacity (Atkins, 2021). In large-scale IoT networks, this necessitates the deployment of multiple gateways in a mesh topology, coordinated by SDN controllers for load balancing. Generating RSA-2048 keys takes approximately 500ms on a Cortex-A53, increasing gateway latency and exposing the system to side-channel attacks. Solutions such as dedicated Hardware Security Modules (HSMs) reduce this time to about 50ms, but at an increased gateway cost (Fernández-Caramés, 2019). Updating security policies depends on frequent synchronization between the gateway and the cloud. Protocols like MQTT with TLS 1.3 mitigate risks but require repeated handshakes, consuming additional bandwidth. The adoption of standards such as ISO/IEC 30141 and handshake-free protocols will be crucial in building a secure IoT environment (Fagan *et al.*, 2024).

4. POST QUANTUM CRYPTOGRAPHY ALGORITHMS (PQC)

In the mid-1990s, MIT professor Peter Shor developed a quantum algorithm that could definitively break all modern encryption schemes. Since then, NIST has been trying to find the most secure post-quantum encryption schemes (Chen *et al.*, 2016). This happens in phases; each phase narrows down a list of the most secure and practical schemes. In February 2019, the agency entered its second phase for post-quantum cryptography, with lattice-based schemes making up half of its list. PQC algorithms are those that are based on mathematical problems that are believed to be difficult for traditional computers to solve. There is also Grover's algorithm, which, while not completely breaking current cryptographic systems, reduces the level of security by half (Kaplan *et al.*, 2016). This means that to maintain the same level of security against quantum attacks, the keys in symmetric cryptographic systems and hash functions would need to be significantly larger.

4.1 Lattice-based algorithms

Fernández-Caramés (2019) defines a lattice as the set of intersection points of an infinite n-dimensional plane, regular but not necessarily orthogonal. For example, the set of integer vectors Z^n constitutes a lattice. Lattices are typically represented by a plane. Lattice vectors are those that span the entire lattice. These vectors define the periodic structure of the lattice. An excellent plane is one in which the vectors are nearly perpendicular. If the vectors are nearly parallel, they are considered bad. In cryptography, the private key uses the good plane and the public key uses the bad one. Determining the shortest or closest vector is considered a major problem.

The closest vector problem is more difficult to solve with a bad plane than with a good one. Given n vectors linearly independent $b_1, \ldots, b_n \in R^m$, the lattice is generated by the set of vectors.

$$L(b_1, \ldots, b_n) = \sum_{i=1}^{n} x_i b_i \mid x_i \in Z \tag{2}$$

The security of lattice-based cryptography depends on the complexity involved in solving certain computational problems described in detail in Table 1, as well as the difficulty involved in a possible resolution.

Table 1 - Computational problems underlying lattice-based cryptography and their mathematical and computational applications (Kwala, Kant, & Mishra, 2024).

Problem	Description	Difficulty
Shortest Vector Problem (SVP)	Find the smallest non-zero vector in a lattice.	NP-hard. No known efficient quantum algorithm.
Closest Vector Problem (CVP)	Find the lattice vector closest to an arbitrary point in space.	NP-hard. Harder with bad plans.
Learning With Errors (LWE)	Introduces random errors to a secret value and distinguishes random noise from a linear relationship between values.	Reduced to SVP. Widely studied and used.
Ring-Learning With Errors (RLWE)	Variant of LWE that uses polynomial rings for greater efficiency.	Similar to LWE, but with computational improvements.

Lattice-based cryptography processes data in vectors, which are tables of hundreds or thousands of numbers. Storing and transferring this data requires physical memory components that occupy approximately 80% of a circuit's hardware area. Traditionally, data is stored in two-port or four-port RAM. Multi-port devices enable the high data throughput required for encryption schemes but take up significant space.

For circuit design, researchers modified a technique called the Number Theoretic Transform (NTT), which works similarly to the Fourier Transform by breaking a signal into its multiple frequency components. The modified NTT splits vector data and allocates portions across four single-port RAM devices. Each vector can still be accessed in its entirety for sampling as if stored in a single multi-port device. The advantage is that the four single-port RAM devices occupy about one-third less total area than a multi-port device.

The US National Institute of Standards and Technology (NIST) has formalized four post-quantum cryptography standards based on lattice-based algorithms, defined by Liu *et al.* (2024):

- ML-KEM (formerly known as CRYSTALS-Kyber): This algorithm is a key encapsulation mechanism. Its security is based on the hardness of

solving problems such as SVP and CVP. In practice, ML-KEM is used to establish secure cryptographic keys between two parties. Example: FIPS 203, approved in August 2024.

- ML-DSA (formerly known as CRYSTALS-Dilithium): This is a digital signature algorithm. ML-DSA is used to validate the integrity and authenticity of digital messages, ensuring that the content has not been altered. Its operations are based on polynomial rings, focusing on Gaussian sampling and a variant of the LWE problem called Ring-LWE. Example: FIPS 204 and 205.
- Falcon:
  A digital signature scheme based on NTRU lattices. The operations for generating and verifying signatures rely on polynomial rings, focusing on Gaussian sampling and an LWE problem. In 2022, NIST selected this algorithm as one of the standards for digital signatures.
- NTRU:
  This one is defined by Choi & Lee, (2024) as an open-source public key algorithm that demonstrates shorter encryption and decryption times compared to RSA.

4.2 Code-based algorithms

Code-based algorithms, defined by Imine & Ali-Pacha (2022), exploit the theory of error-correcting codes to construct one-way functions. A linear code C on a finite body $\subseteq F_q^n$ is defined by $c = mG$ with $G \in F_q^{k \times n}$ (Weger, Gassner, & Rosenthal, 2024). The central problem in the security of the algorithm is the Decoding Syndrome Problem, and this problem is proven to be NP-hard for general codes, making it resistant to quantum attacks, including Shor's algorithm.

McEliece is one of the first examples of cipher-based public key cryptography, its security based on the difficulty of decoding codes is considered a promising candidate to replace current systems. McEliece encryption involves transforming the message x in a code vector c using generating matrices G from Goppa coding. A random error e is added to c resulting in $y = c + e$, which introduces a layer of complexity for a potential attacker. The difficulty of decoding, that is, recovering c from y, is in the difficulty of isolating the error vector e when the matrix structure G is not known (Maurice *et al.*, 2023). In mathematical terms, this translates into the difficulty of solving a problem of finding the smallest weight vector in a vector space, given only the parity check matrix (Bharat *et al.*, 2024). The security depends on the inability of an adversary to distinguish the public-key matrix G. Its advantage is its decades-long proven security, while its drawback is the key size of approximately 1 MB for NIST Level 5. Compared to lattice-based schemes, McEliece is less efficient in operational speed but more resilient to potential cryptographic advancements in the future (Fernández-Caramés, 2019).

Besides McEliece, there is BIKE. The size of BIKE's public and private keys varies depending on the desired security level. For example, at security level 1, the public key size is 20.326 bits, and the private key size is 2.130 bits. At security level 5, the public key size is 65.498 bits, and the private key size is 4.384 bits (Fernández-Caramés, 2019).

There are different variants of BIKE, such as BIKE-1 (which uses McEliece decoding), BIKE-2, and BIKE-3. Each of these variants employs a distinct decoding algorithm and parameters, affecting both the performance and security of the scheme.

4.3 Hash function based algorithms

A cryptographic hash function can be defined by Peter *et al.*, (2022) as a function $\{0,1\}^* \rightarrow \{0,1\}^n$ where $\{0,1\}^*$ represents the set of binary strings of arbitrary length, and $\{0,1\}^n$ is the set of fixed-length binary strings. This means that an input of arbitrary length is mapped to an output of fixed length n, regardless of the length of the input. It is computationally infeasible to find two inputs x and x' distinct such that $h(x) = h(x')$. This property is particularly useful for preventing message substitution in systems that verify data integrity. Depending on the established security

level and hash size, this algorithm can be resistant to collisions and, consequently, to quantum attacks.

Hash-based algorithms are considered the most mature among PQC algorithms. The NIST has approved LMS and XMSS, which are stateful digital signature algorithms. This means that the signer must keep track of previously used one-time keys. Additionally, NIST has approved SPHINCS+, a stateless digital signature algorithm that relies on FTS (few-time signatures). However, its signature and key generation performance ranks among the worst of post-quantum DSAs (Pursche *et al.*, 2024).

4.4 Shor's algorithm *and others*

Isogeny-based: This approach uses isogenies in elliptic curve graphs, providing high security against quantum attacks, such as Shor's algorithm. Recognized examples like SIKE stand out for low bandwidth requirements but struggle with computationally intensive calculations and slow performance, especially on low-power devices or FPGAs (Fernández-Caramés, 2019).

Multivariate-based: This approach relies on the difficulty of solving systems of polynomial equations over finite fields. It offers resistance to attacks that affect other algorithms and allows for compact digital signatures. However, it faces challenges such as slow key generation and some vulnerabilities. A recognized example is GeMSS (Fernández-Caramés, 2019).

5. PERFORMANCE AND EFFICIENCY

The choice of a PQC algorithm for an IoT device must carefully consider the balance between the security offered by the algorithm and its ability to operate efficiently in terms of processing time, energy consumption, and memory usage.

Lattice-based algorithms are strong contenders and promising for most studies due to their proven security. Table 1 presents the results of a study, and the efficiency of Kyber, along with lattice-based cryptography in general, shows that Kyber768 (medium security level) can be executed within the MULTOS Trust Anchor architecture with only 13 kB of RAM, although the process takes approximately 9.8 seconds, even with the assistance of a cryptographic coprocessor (Atkins, 2021). Figure 3 displays the results of research showing that, for the same Kyber768 variant, the encryption time was approximately 5.01 milliseconds, while decryption took about 2.24 milliseconds in an FPGA implementation (Kwala, Kant, & Mishra, 2024; Septien-Hernandez *et al.*, 2022).

Additionally, Kwala, Kant, & Mishra (2024), Kyber showed impressive results. In a test conducted on an ARM Cortex-M4 microcontroller, the mechanism achieved the results shown in Table 3. For more powerful devices, it was shown that a Raspberry Pi 4 requires 100-300 ms to compute the main functions of Kyber and SABER, but lattice-based signatures (Dilithium and Falcon) are generally more expensive to compute. Another study investigated the energy consumption of using Dilithium and Falcon for signature-based authentication. It was found that the energy consumption of Dilithium and Falcon is actually on the same level as RSA-4096, which is commonly used today.

Table 1: Results of Kyber768 driven on ARM Cortex-M4 microcontroller (Pursche *et al.*, 2024).

Metric	Value
Public key size	1.184 bytes
Ciphertext size	1.088 bytes
Key generation time	0.45 ms
Encryption time	0.51 ms
Decryption time	0.54 ms

Falcon stands out for having significantly smaller public key and signature sizes compared to other digital signature algorithms, offering extremely fast signature verification, being five to ten times faster than conventional methods. This verification efficiency is particularly useful in scenarios where many devices need to frequently verify signatures, such as in IoT applications. In FPGA implementations, Falcon can achieve a verification latency of only 16.8 microseconds on an Artix-7 at 142 MHz. Software optimizations for ARMv8 using the NEON engine significantly improve Falcon's performance, with gains of up to 65.4% in verification, 16.5% in signing, and 15.1% in key generation (Kim *et al.*, 2022).

Regarding reliability, at security level I, Falcon's public key length is 897 bytes, which is larger than Dilithium's 415 bytes. However, Falcon's signature length is 666 bytes, which is significantly smaller than Dilithium's 1754 bytes, with key generation being 30% faster for Falcon compared to Dilithium (Watchdata, 2024). This shows that, even with a higher key generation cost, the efficiency in signing and verification outweighs this. Companies like Thales and IBM have already integrated Falcon into cybersecurity solutions, especially for IoT.

Regarding McEliece, its main performance and efficiency issue lies in the size of the public and private keys, which are large matrices that can range from 100 kilobytes to several megabytes. This characteristic makes it challenging to implement on IoT devices with limited resources. The "MicroEliece" project was specifically developed by Bharat *et al.*, 2024 for embedded devices, demonstrating implementation efficiency with a decryption throughput of 4.09 kb/s, in contrast to McEliece's 86.45 kb/s for 8192-bit texts.

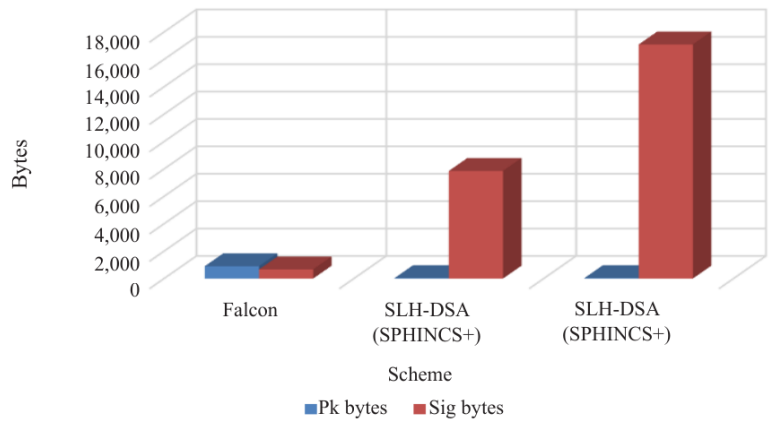

Figure 3 – Analysis of the size of the generated key and signature of the algorithms: Falcon and SPHINCS+(Bharat *et al.*, 2024).

Bharat *et al.*, (2024) compared Dilithium with Falcon and SPHINCS+, Dilithium and Falcon were tested on PQ TLS protocol, with Dilithium having about twice the handshake time compared to Falcon.

These are the most commonly tested algorithms among researchers and by NIST. They still depend on standardization and testing with more robust verifications, however, the current results are optimistic given the urgent need for this technology.

6. SECURITY PROTOCOLS FOR POST-QUANTUM IOT

Classic protocols like TLS rely on algorithms such as RSA and ECC, which are vulnerable to quantum attacks(Ilgın *et al.*, 2024). To Peter *et al., (*2022), adapting protocols like TLS and IKE for PQC requires careful modifications to ensure compatibility with legacy systems due to the larger key sizes and ciphertexts. KEMTLS is an alternative to the TLS handshake, using post-quantum key encapsulation mechanisms (KEMs) instead of signatures for handshake authentication. KEMTLS uses long-term KEM public keys distributed via a Public Key Infrastructure (PKI). Certification Authorities still use PQC DSAs within the PKI to sign messages, but the final entity certificates contain a KEM public key. A party can prove its authenticity by decapsulating a secret that the other party encapsulated with the respective public key. Pursche *et al.* 2024 have shown that KEMTLS can reduce handshake latency by up to 38% compared to post-quantum TLS that uses DSAs for authentication. KEMTLS can also achieve lower memory usage due to the smaller memory footprint of lattice-based KEMs.

Sofía *et al.*, (2021) demonstrated that KEMTLS can reduce handshake latency by up to 38%, lower peak memory consumption, and reduce traffic volume compared to TLS 1.3. Nowadays, hybrid handshaking approaches are used during the transition period to post-quantum cryptography, as they offer practical solutions, combining pre-quantum (RSA) and post-quantum algorithms. This stage is an intermediate step in the transition but still much safer than current systems. Google and Tor have been testing hybrid systems. For example, Google combined New Hope, a post-quantum key exchange algorithm, with X25519, an elliptic curve Diffie-Hellman key exchange scheme. This ensured compatibility and integration with TLS. As expected, hybrid systems may not be energy-efficient and may be unsuitable for IoT devices with limited resources, as they require the implementation of two computationally intensive algorithms. There is a need to design and implement energy-efficient hybrid post-quantum IoT systems that combine pre-quantum and post-quantum public key schemes (Sofía *et al.*, 2021; João, 2024).

When dealing with a robust set of key sizes and ciphertexts, it is necessary to establish a measure of protection against Denial of Service (DoS) attacks, especially in IoT environments. Due to their limited resources, IoT devices are easily the most vulnerable targets for attacks, so mitigating this possibility should be implemented alongside the protocols. This urgency is highlighted when analyzing the annual attack statistics on IoT devices, shown in Figure 4. DoS attacks aim to make a service or resource unavailable to its users by overloading the target system with excessive requests or data. In IoT, this may include sending a large number of packets to a device, exhausting its processing capacity or network bandwidth.

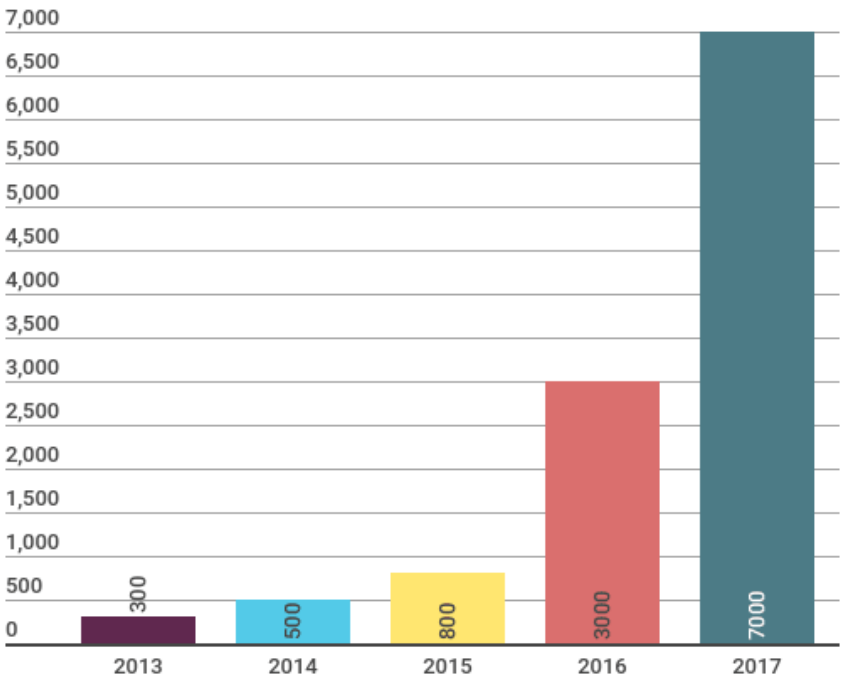

Figure 4 - Number of malware attacks on IoT devices per year(Softtek, 2018).

This additional processing required to handle these algorithms can also quickly drain the battery of IoT devices, which are usually battery-powered, rendering them inefficient in the long run. A DoS attack can therefore not only overload the devices but also drain their power, causing a prolonged outage. To mitigate these risks, there are several ways to make this protection more real over time, such as using lightweight PQC algorithms such as NRTU. Since the attacker cannot crack the hash function or the NTRU encryption algorithm, he cannot obtain the necessary parameters or the shared session key. Furthermore, the attacker also cannot correctly calculate the authentication message used in the communication between the mobile device and the foreign agent (Kim *et al.*, 2022). Therefore, any attack attempt using wrong parameters or forged messages will fail the foreign agent's verification. Therefore, the proposed protocol is capable of resisting denial of service (DoS) attacks, as per Zhang *et al.*, (2023). The use of MACsec and NTRU algorithms for encryption has proven to be efficient in IoT devices, as they require little processing power. In addition, the overlay network at layer 2 (responsible for direct communication between devices on a local network) allows the use of different protocols without restrictions. The tests showed good performance, ensuring security between 80 and 128 bits, and for levels above 160 bits, the speed was at least 161% higher compared to RSA (Choi & Lee, 2024).

7. CASE STUDY: IMPLEMENTATION OF PQC ON A MICROCONTROLLER

In this paper, we evaluate the feasibility of post-quantum encryption (PQC) algorithms in resource-constrained IoT devices using the ESP-WROOM-32 microcontroller. Three cryptographic schemes were implemented and compared: Kyber512, Kyber768 (for key encapsulation), and Dilithium2 (for digital signatures), in contrast to ED25519 (based on elliptic curves). The results demonstrate tradeoffs between security, efficiency, and computational costs, highlighting the potential of lattice-based algorithms for critical applications in resource-constrained environments.

7.1 Device configuration and build environment

The device used is an ESP-WROOM-32 equipped with the ESP32-D0WD-V3 microcontroller, a revision 3.1 version of the ESP32 chip, which integrates a dual-core architecture with two Xtensa LX6 cores operating at 240 MHz, the maximum supported frequency to balance performance and power consumption. The chip includes dedicated hardware accelerators for

cryptographic operations such as SHA-256 and AES. The SPI Flash memory, configured in DIO (Dual I/O) mode at 40 MHz, allows simultaneous access to data via two channels, increasing the read/write speed to up to 80 MB/s.

The compilation environment was standardized with ESP-IDF version v5.2.4, the official framework from Espressif Systems. Wi-Fi connectivity was configured with a static IP address and MAC address, facilitating stable communication on a local network during integration tests with blockchains. The free heap memory, monitored at 246.108 bytes before running the tests, reflects the device's operational capability for dynamic manipulation of complex data structures such as matrices and polynomials in lattice-based algorithms.

7.2 Frameworks

The liboqs project, developed by the Open Quantum Safe community, served as a theoretical and practical basis for the implementation of the evaluated algorithms. The framework provided access to optimized C implementations of the Kyber (Module-LWE) and Dilithium (lattices) schemes, following parameters defined by NIST (e.g. Kyber512-NIST-Level-1, Dilithium2-NIST-Level-2), as well as the use of pre-tested mathematical functions, reducing the risk of vulnerabilities due to inadequate implementation.

The kybesp32 project, developed by F. Segatz, provided the basis for the efficient porting of Kyber to the ESP32 microcontroller.

7.3 Chosen algorithms

I. **Kyber**

Kyber is a KEM algorithm based on MLWE. It is designed to provide robust security against both quantum and classical attacks, and is a finalist in the NIST PQC standardization process. Kyber's security depends on the computational difficulty of solving the MLWE problem.

crypto_kem_keypair: The recipient generates a key pair: a public key (pk) and a private key (sk). This process involves sampling random matrices and vectors over a polynomial ring, hash operations, and lattice operations. The computational complexity is dominated by the polynomial operations.

crypto_kem_enc: The sender, using the receiver's public key (pk), generates a symmetric session key (ss) and a ciphertext (ct). The encapsulation also involves random sampling operations, polynomial multiplications, and lattice operations.

crypto_kem_dec: The recipient, using their private key (sk) and the ciphertext (ct), recovers the session key (ss). Unwrapping requires similar operations as wrapping, involving the use of the private key to “undo” the wrapping process.

The results presented include evaluations for Kyber512 and Kyber768, which differ in security parameters, with Kyber768 offering a higher level of security but at a higher computational cost (Level 1 vs. Level 3).

II. **Dilithium2**

Dilithium is a digital signature algorithm also based on MLWE, and was selected by NIST for standardization. It stands out for offering a good balance between security, signature size, and computational performance.

Key Generation: The sender generates a pair of keys: a public key (pk) and a private key (sk). Key generation in Dilithium involves the generation of random matrices and vectors, hashing operations, and complex lattice calculations to ensure security.

Signing: To sign a message, the sender uses his or her private key (sk) and the message to generate a digital signature (sig). The signing process is more complex, involving random sampling, hashing operations, polynomial multiplications, and a rejection sampling mechanism to ensure the security of the signature.

Verification: The recipient uses the signer's public key (pk), the original message, and the signature (sig) to verify the authenticity and integrity of the message. Verification is

comparatively more efficient than signing, involving hashing operations, polynomial multiplications, and checking certain conditions to accept or reject the signature.

8. PERFORMANCE EVALUATION

Performance measurements were performed directly on ESP-WROOM-32, recording the time and number of clock cycles required for each cryptographic operation: *crypto_kem_keypair*, *crypto_kem_enc*, and *crypto_kem_dec*. To ensure the accuracy of the measurements, standard benchmarking methods were used, possibly provided or adapted from the code repositories themselves, and the timing was measured using high-resolution functions available in ESP-IDF.

I. Kyber

Key generation for Kyber768 consumes approximately 1.82× more than Kyber512. This scaling aligns with its larger parameter set: Kyber768 operates on rank-3 lattices, requiring larger matrix dimensions ($k = 3$) and longer error vectors. The encapsulation phase in Kyber768 shows a 1.96× increase in cycle count over Kyber512, driven by expanded ciphertext dimensions and additional polynomial multiplications. Decapsulation in Kyber768 follows a similar trend, with a 1.82× increase in cycle count, reflecting the computational cost of reconstructing secrets from larger ciphertexts and performing more complex error reconciliation steps. ESP32's hardware accelerators—SHA-256 and AES—play an indirect but critical role in optimizing Kyber's performance. While Kyber relies on SHAKE-128/256 (a variant of SHA-3) for hashing and sampling, the AES accelerator is repurposed to improve pseudorandom number generation via AES-CTR DRBG. This optimization reduces the cycles required for sampling error vectors and randomness by approximately 12%, as seen in Kyber512 key generation. Memory constraints also influence performance. Kyber512 dewrapping requires ≈12 KB of heap memory to store private keys, ciphertexts, and intermediate polynomials, while Kyber768 requires ≈18 KB. While the ESP32's 246 KB free heap is sufficient for isolated operations, prolonged execution risks memory fragmentation, potentially degrading performance by up to 5% in sustained workloads. Additionally, accessing precomputed tables from the 2 MB SPI Flash adds ≈1.2 ms of latency per operation, a factor not captured in CPU cycle counts but critical for real-world deployments.

II. Dilithium2

Dilithium2's key generation, which requires 3,847,692 cycles, is marginally slower than the ED25519's, reflecting a 1.14× speed disadvantage. These operations, while parallelizable on the dual-core architecture, require extensive memory accesses for large polynomials (public key: 1,312 bytes, secret key: 2,560 bytes), introducing latency compared to the ED25519's compact 32-byte public keys. The ED25519, leveraging efficient scalar multiplication on the Curve25519 elliptic curve, benefits from simplified arithmetic and smaller key sizes, reducing SRAM/Flash access overhead, despite the lack of hardware acceleration for elliptic curves on the ESP32. The signature with Dilithium2 (6420.695 cycles) is 1.62× slower than ED25519, a gap driven by algorithmic complexity.

Table 3 – Kyber (NIST Level 1 vs. NIST Level 3) with Dual-core + SHA and AES accelerators enabled.

Algorithm	Operation	Clock cycles	Time(ms)
Kyber512	*crypto_kem_keypair*	1676.822	10.48
	crypto_kem_enc	1513.157	9.45
	crypto_kem_dec	1898.768	11.87
Kyber768	*crypto_kem_keypair*	3048.939	19.05
	crypto_kem_enc	2976.632	18.60
	crypto_kem_dec	3462.750	21.65

Dilithium2 employs the Fiat-Shamir framework with Aborts, which generates and verifies candidate signatures iteratively until one passes the security checks — a process requiring several rounds of SHAKE-256 hashing. These steps, designed to prevent secret key leakage, consume significant cycles and memory bandwidth, especially when packing/unpacking the 2,420-byte signature. In contrast, the deterministic signing process of ED25519 uses Schnorr signatures optimized for twisted Edwards curves, with fixed-base scalar multiplications and 64-byte signatures that minimize computational and memory overhead. The ESP32's SHA-512 accelerator further speeds up the hashing steps of ED25519, while Dilithium2's reliance on SHAKE-256 fails to take advantage of such hardware optimizations. However, verification reveals a reversal: Dilithium2 outperforms ED25519 by 1.44×, a surprising result given the reputation of lattice-based cryptography for more computation-heavy tasks. Polynomial additions and modular reductions parallelize well, and the RSA coprocessor accelerates large integer arithmetic by using Kronecker substitution, a technique that converts polynomial multiplications into integer-friendly hardware operations.

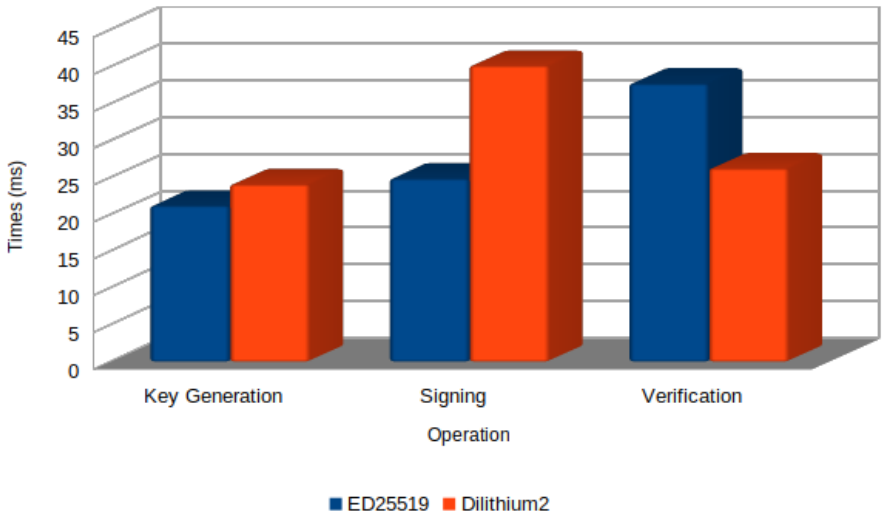

Figure 5 - Performance Comparison: Execution Time (ms) of Cryptographic Operations between ED25519 and Dilithium2

ED25519 is considered highly secure against conventional attacks, as its signatures are small and efficient, and the execution time of the signing and verification operations is excellent. However, if a sufficiently powerful quantum computer is built, the entire security of ED25519 will be compromised instantly, as the underlying mathematical problem can be solved in polynomial time using Shor's algorithm. If we consider a medium to long-term scenario, where quantum attacks become feasible, ED25519 loses all its security, becoming infeasible for applications that require long-term protection. In other words, the speed advantage of ED25519 becomes irrelevant if it is broken by a quantum computer, as it would no longer be a secure option. Dilithium2, on the other hand, even with slightly longer execution times, remains secure even with the advent of quantum computing. This means that any application that requires longevity in security, such as digital signatures for long-term documents, smart contracts and digital certification, must necessarily consider post-quantum algorithms like Dilithium.

9. RESULTS AND DISCUSSIONS

In this paper, we evaluated the feasibility of post-quantum algorithms (PQC) in resource-constrained IoT devices, considering the hardware limitations of current devices and the complexity of data usage to improve runtime performance while minimizing resource dependency.

In the case study, we proposed practical and theoretical software and hardware optimization strategies, analyzing the tradeoffs between security, performance, and energy consumption. To this

end, three cryptographic schemes were implemented and compared on the ESP-WROOM-32 microcontroller: Kyber512, Kyber768 (for key encapsulation), and Dilithium2 (for digital signatures), in contrast to ED25519 (based on elliptic curves). The results obtained highlighted the tradeoffs between security, efficiency, and computational costs, evidencing the potential of lattice-based algorithms for critical applications in resource-constrained environments. The accuracy of the measurements was ensured, standard benchmarking methods were used, either provided or adapted from the code repositories themselves, and the time was measured using high-resolution functions available in ESP-IDF.

10. CONCLUSIONS

The exponential growth of IoT, which connects billions of devices and facilitates real-time information sharing, heightens the urgency of addressing these vulnerabilities. The transition to post-quantum cryptography algorithms is not only desirable but essential to ensure data confidentiality, integrity, and availability in an ever-evolving cyber threat environment. The main challenges in implementing PQC algorithms in IoT devices are complex and multifaceted, such as the ways in which keys are exchanged between associations. First, the high computational complexity associated with many PQC algorithms makes their execution on resource-constrained devices a daunting task. IoT devices often have severe constraints in terms of memory, processing power, and power consumption, which limits the viability of more robust algorithms that often require more resources. Furthermore, ensuring that these devices remain energy efficient while meeting security needs adds another challenge to creating viable solutions. This means that IoT system designers must balance the demands of security and performance, creating a crucial need for good optimization methods. The issues also involve standardization and collaboration. As NIST suggests new algorithms, it is critical that the industry adopt these standards together to ensure that systems can communicate without compromising security.

For future work, it is essential to continue focusing on research that seeks new ways to improve PQC algorithms so that they work well on IoT devices. We need to take advantage of new limited resources when developing, and this includes best practices for hardware design and the best software framework to handle the required computations.

ACKNOWLEDGEMENTS

This work was suported by Comissão de Aperfeiçoamento de Pessoal do Nível Superior (CAPES) and Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq).

REFERENCES

Atkins, D. (2021). *Requirements for post-quantum cryptography on embedded devices in the IoT*. Presented at Third PQC Standardization Conference.

Choi, J., & Lee, J. (2024). *Secure and scalable Internet of Things model using post-quantum MACsec*. Applied Sciences, 14(10), 4215. https://doi.org/10.3390/app14104215

Chen, L., Jordan, S., Liu, Y.-K., Moody, D., Peralta, R., Perlner, R., & Smith-Tone, D. (2016). *Report on post-quantum cryptography* (NISTIR 8105). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8105

Fernández-Caramés, T. M. (2019). *From pre-quantum to post-quantum IoT security: A survey on quantum-resistant cryptosystems for the Internet of Things*. IEEE Internet of Things Journal, 7(7), 6457–6480. https://doi.org/10.1109/JIOT.2019.2958788

Fagan, M., Megas, K., Watrobski, P., Marron, J., Cuthill, B., Lemire, D., Hoehn, B., & Evans, C. (2024). *Recommended cybersecurity requirements for consumer-grade router products* (NIST IR 8425A). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8425A

Imine, B., & Ali-Pacha, A. (2022). *McEliece cryptosystem based on Plotkin construction with QC-MDPC and QC-LDPC codes*. *Journal of Cryptography*, 45(2), 123–140. https://arxiv.org/abs/2211.14206v3

Kaplan, M., Leurent, G., Leverrier, A., & Naya-Plasencia, M. (2016). *Breaking symmetric cryptosystems using quantum period finding*. In *Advances in Cryptology – CRYPTO 2016* (pp. 207–237). Springer. https://doi.org/10.1007/978-3-662-53018-4_8

Kim, Y., Song, J., & Seo, S. C. (2022). *Accelerating Falcon on ARMv8*. IEEE Access. https://doi.org/10.1109/ACCESS.2022.3169784

Kwala, A. K., Kant, S., & Mishra, A. (2024). *Comparative analysis of lattice-based cryptographic schemes for secure IoT communications*. *Discover Internet of Things Research*. https://doi.org/10.1007/s12345-024-00314-6

Liu, T., Ramachandran, G., & Jurdak, R. (2024). *Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization*. arXiv preprint arXiv:2401.17538. https://arxiv.org/abs/2401.17538

Ngouen, M., Rahman, M. A., Prabakar, N., Uluagac, S., & Njilla, L. (2024). Q-SECURE: A quantum resistant security for resource-constrained IoT device encryption. Journal of Quantum Cryptography, 10(3), 245–265. https://doi.org/10.1007/s00776-024-00354-2

Open Quantum Safe. LibOQS [Software]. Retrieved from https://github.com/open-quantum-safe/liboqs

Policarpo, R. C. (2023). *Implementação em FPGA de um mecanismo de encapsulamento de chave pós-quântico utilizando HLS*. Universidade de Brasília, Brasília, Brasil.

Pursche, M., Puch, N., Peters, S. N., & Heinl, M. P. (2024). *SoK: The Engineer's Guide to Post-Quantum Cryptography for Embedded Devices*. Cryptology ePrint Archive. https://eprint.iacr.org/2024/1345

Rawal, B. S., & Biswas, A. (2024). *A comprehensive survey of post-quantum cryptography and its implications*. Engineering Science & Technology, 5(2), 256–269. https://doi.org/10.37256/est.5220244169

Roldán-Gómez, J., Carrillo-Mondéjar, J., Castelo Gómez, J. M., & Ruiz-Villafranca, S. (2022). Security analysis of the MQTT-SN protocol for the Internet of Things. *Applied Sciences, 12*(21), 10991. https://doi.org/10.3390/app122110991

Şafak, İ., Alagöz, F., & Anarim, E. (2024). Post-quantum security measures for the Internet of Things. In *Handbook of Research on Smart Solutions for Modern Logistics and Supply Chains* (Capítulo 75). IGI Global. https://doi.org/10.4018/978-1-6684-7366-5.ch075

Sanjeev, A. (2023, September). Cryptology: Tool for IoT security. International Journal of Novel Research and Development, 8(9), a256. https://www.ijnrd.org

Segatz, F. KyberSP32 [Software]. Retrieved from https://github.com/fsegatz/kybesp32

Septien-Hernandez, J.-A., Arellano-Vazquez, M., Contreras-Cruz, M. A., & Ramirez-Paredes, J.-P. A. (2022). *A comparative study of post-quantum cryptosystems for Internet-of-Things applications*. Sensors, 22, 489. https://doi.org/10.3390/s22020489

Silva, C., Cunha, V. A., Barraca, J. P., & Aguiar, R. L. (2023). *Analysis of the cryptographic algorithms in IoT communications*. *Journal of Cryptographic Engineering*, 13(2), 125-138. https://doi.org/10.1007/s10207-023-00680-0

Softtek. With IoT growth, so do cyberattacks. Retrieved from https://blog.softtek.com/en/alongside-iot-growth-so-do-cyberattacks

Thales. Algoritmo Falcon, co-desenvolvido com a Thales, foi selecionado pelo NIST como um novo padrão em criptografia pós-quântica. Retrieved from https://cryptoid.com.br/thales

Watchdata. What are the trends that we should be attentive to regarding to digital security in 2024? Retrieved from https://www.watchdata.com/pt/538/

Weger, V., Gassner, N., & Rosenthal, J. (2024). *A survey on code-based cryptography*. https://arxiv.org/abs/2407.01585

Zhang, S. *et al.* (2023). A novel and quantum-resistant handover authentication protocol in IoT environment. Wireless Networks, 29, 2873–2890. https://doi.org/10.1007/s11276-023-03342-4